Course: Management of AI Products (MBAZG583)

Student Name: Mayur H. Borate

Student ID: 2025MB26174

LECTURE REFLECTIONS for sessions 9 to 12

➢ Lecture 9 - AI Product Development Cycle

- Development of AI products should be stage-gated rather than moving directly from an interesting idea to building a full product.
  - Broad cycle: Discover/Frame -> Prototype -> Pilot -> MVP -> Scale and iterate. Each stage should reduce a different uncertainty before further investment.
  - Discovery proves that the user problem is worth solving; prototype proves concept/workflow/feasibility; pilot checks actual usage in a limited setting; MVP tests repeatable value with real users; scale tests economics and reliability.
- Product development stages need different evidence and not just different levels of technical completeness.
  - Evidence: user pain and frequency, user reaction to prototype, adoption intent, workflow fit, data/technical feasibility, actual usage, operational readiness, and a clear next decision.
  - Figma-to-production journey for AI copilots and Notion AI examples show why a convincing demo does not itself establish production viability.
- Prototype - a low-cost way to test an assumption. Its main purpose is learning; it need not be fully functional or production-ready.
  - Concept prototype - tests whether the idea/problem and value proposition make sense to the user.
  - UX prototype - tests how the AI should appear to the user: assistant, inline suggestion, background support, etc. (Notion AI vs Microsoft Copilot as a workflow example).
  - Workflow prototype - tests the end-to-end steps, human intervention points, and whether the AI removes actual friction rather than adding another screen.
  - Data prototype - tests whether useful, lawful, fresh and representative data is available; also checks what minimum event tracking will support later measurement.
- Prototype type should be chosen based on the biggest unknown, not based on which prototype looks most impressive.
  - For early uncertainty, a paper/mock-up or simulated experience may teach more than an expensive model. Increase fidelity only when the earlier assumptions are reasonably proven.
  - Useful measures: speed of learning, clarity of customer feedback, reaction to the workflow, and whether the prototype removes the riskiest assumption.
- Prototype vs MVP (Minimum Viable Product) - both are learning tools but they have different exposure and commitments.
  - Prototype = Is this idea or interaction worth pursuing? MVP = Can a limited but usable offering create real value and change user behavior?
  - MVP reaches actual users and therefore requires sufficient reliability, privacy, safety, support, and clear success/failure criteria - even when feature scope is deliberately small.
  - The failure is to call everything an MVP and end up confusing internal learning with a product that has actually earned user trust.

- Different MVP approaches can be used without building the complete AI solution from day one:
  - Wizard of Oz MVP - experience looks automated, but a human performs some work behind the scenes. Good for testing demand, but manual cost and disclosure/trust implications need to be understood.
  - Concierge MVP - users know that they are receiving a customized human-assisted service. Helps understand how users behave before automation.
  - Partial automation MVP - only a few repeatable steps are automated; complex, uncertain, or risky tasks are reviewed manually.
  - Workflow assistant MVP - AI is introduced at one high-friction point in the existing task rather than building an independent AI interface.
  - Choosing an MVP needs to consider manual effort per user, learning per rupee spent, user trust, repeat use, and whether the arrangement can eventually scale.
- AI illusion techniques - a product can appear intelligent even when substantial work is manual. This is useful for early validation but dangerous if the operating model is hidden from the team.
  - Human-in-the-loop behind the scenes, staged automation, manual fallback, and learning before full automation are different ways of reducing early risk.
  - Strong staged automation defines: what humans do now -> what will be automated next -> evidence/threshold needed -> owner and target economics.
  - Look at queue load, reviewer time, override rate, cycle time, quality consistency, and cost per successful task before expanding automation.
  - If manual support cannot be reduced economically, the business case of the so-called AI product needs to be revisited.
- Development governance artifacts help avoid progressing only because a demo is attractive:
  - Prototype brief - problem, target user, key uncertainty, cheapest test, expected learning and decision owner.
  - MVP scope memo - audience, value proposition, in-scope/out-of-scope functions, human support, risks and operating limits.
  - Evidence threshold review - what was measured, minimum pass criteria, adoption/workflow evidence, and issues not yet resolved.
  - Go / No-go / Iterate memo - recommendation, trade-offs, unresolved risks, approver and next evidence check.
- No-code prototype of an AI Sales Copilot example - first validate whether users value the suggested insights and workflow; build AI integration only after learning what must actually work.
- PM responsibility is to decide what must be proven before spending more, when human support is acceptable and when the product is ready for a controlled launch.

➢ Lecture 10 - Experimentation, Metrics, and Evaluation

- AI products should not be evaluated only through the conventional software metrics of feature completion, uptime, or usage. Model performance must translate into user value and business outcomes.
  - Offline evaluation - test model behavior on held-out/reference data before release. Online evaluation - check user behavior, trust, workflow completion and actual business impact after release.
  - A better offline score need not mean a better product. A small improvement on an average can also hide major deterioration for a critical user segment.
- Metric selection is a product decision because it determines the trade-off that the team will optimize.

        - Example: Netflix recommendation ranking needs to improve discovery/retention and not just clicks; Google Search quality needs to consider real user satisfaction.
        - Metrics should be broken down by cohort, error type, review burden, and long-term impact instead of relying on only the overall average.
    - Confusion Matrix - compares actual and predicted classification outcomes:
        - True Positive (TP) and True Negative (TN) are correct classifications; False Positive (FP) and False Negative (FN) are the two kinds of mistakes.
        - Accuracy = (TP + TN)/(TP + TN + FP + FN). Useful as a general measure, but misleading when positive events are rare or when FP and FN have very different consequences.
        - Precision = TP/(TP + FP). Of all the positive predictions made, how many were actually correct?
        - Recall = TP/(TP + FN). Of all the real positives, how many did the model identify?
        - F1-score = 2 x (Precision x Recall)/(Precision + Recall). Harmonic mean useful for balancing precision and recall, but not a substitute for analyzing business costs.
        - Changing the classification threshold normally changes precision, recall, review load, and the costs of each kind of error.
    - Class imbalance and consequences matter more than a large-looking accuracy number.
        - The loan example discussed through the Excel confusion matrix: missing a good loan means opportunity loss; approving a defaulting loan can cause much larger loss. Hence cost of an FP/FN cannot be assumed equal.
        - Stripe Radar / payment fraud: too many false positives block genuine customers; too many false negatives increase fraud losses. Breast cancer screening: missing a genuine case can be more serious than arranging an additional review.
        - Decide the positive class, acceptable error threshold and who bears the cost before choosing the target metric.
    - Evaluation from a business perspective should connect three layers:
        - Model metric - precision, recall, ranking quality, etc.; Business metric - revenue, conversion, prevented loss, cost; Customer/trust metric - complaints, churn, friction, confidence, appeals.
        - Amazon recommendation lift can increase sales but also create returns or trust concerns; PayPal fraud prevention can reduce losses but add customer friction.
        - A dashboard that rewards model score alone can drive harmful optimization. The PM should define an agreed metric pair/trio and a rollback or review trigger.
    - A/B Testing - compare a control group with a treatment group to find whether the AI feature actually causes improvement.
        - Start with a clear hypothesis: what changes, for whom, what success metric is expected and what guardrail must not degrade.
        - Randomization is needed to reduce selection bias. Consider experiment duration, sample adequacy, segment differences, and treatment spillover/contamination.
        - For a recommendation or learning system, results can be noisy due to changing user behavior and feedback loops. A statistically positive aggregate effect should still be checked for harmed cohorts.
        - Examples in the slides include Netflix ranking, Booking.com experimentation culture and Duolingo learning-experience testing.
        - Experiment launch criteria should specify stop rules and safety/quality guardrails, particularly where a change could disadvantage users.

- Feedback loops - user behavior provides data for improving models, but that data may itself be influenced by previous model recommendations.
  - Healthy loop: exposure -> user response/correction -> quality review -> improved model/product -> subsequent monitoring.
  - Risk of self-reinforcing bias: showing only popular items generates more engagement for those items, reducing diversity and making the model increasingly narrow.
  - TikTok engagement and Waze routing are useful examples to consider immediate feedback vs wider, longer-term outcomes.
  - Relevant evidence: feedback diversity, novelty decay, correction rate, long-term user outcomes and whether there are unintended external effects.
- Metric-design activity: a useful product scorecard needs only the measures that change a decision, each with an owner, review cadence, threshold and action.
  - For a digital payments fraud product: fraud loss/prevention + false-decline friction + legitimate customer trust/complaints are complementary measures.
  - A PM should be able to answer: Which metric would justify release, which metric would stop release and which segment might be harmed despite a better overall score?

➢ Lecture 11 - Scaling AI Products

- Scaling AI is not just handling a higher number of API calls. It is growth in users, data/models, workflow complexity, and organizational dependence.
  - A feature which is initially optional may become mission-critical when users incorporate it into their daily work; the same failure now has much higher impact.
  - Evidence for scale: peak load, degraded-mode behavior, support burden, workflow dependence and ability to recover after failure.
- Response-time economics - model quality, inference latency and cost have to be optimized together.
  - Inference cost depends on model choice, input/output token volume, request frequency, context length, infrastructure and support/review cost.
  - Latency affects user experience. A feature can be accurate but unusable if the response takes too long for the actual workflow.
  - Useful signals: P95 latency (95% of requests are faster than this), cost per successful task, abandonment rate, throughput, error rate and output quality by latency band.
  - Capacity planning should consider peak traffic, waiting time, concurrency and the impact of unexpected spikes rather than average load alone.
  - A smaller/faster model can be commercially better than a larger model when the extra accuracy is not valued enough to cover speed and operating cost.
- Unit economics becomes important at scale: Revenue/value per completed task must justify inference + infrastructure + human support + monitoring costs.
  - Before offering broad usage or aggressive pricing, stress-test cost curves and sensitivity to token usage, model switching and peak demand.
  - OpenAI inference economics and Google Translate response-time expectations were examples used to consider quality versus speed.
- MLOps (Machine Learning Operations) is the operational backbone for reliably developing, deploying and maintaining models.

        - Pipeline: data ingestion/validation -> feature preparation -> training/evaluation -> model registry -> deployment -> monitoring/feedback.
        - Automation should cover repeatable testing, data checks, deployment, alerts and rollback; not only retraining.
        - Monitor data/feature drift, model quality deterioration, failed jobs, alert quality, mean time to recover and operational ownership.
        - Spotify deployment and Amazon SageMaker automation examples show that scalability needs repeatable systems, not manual heroics.
    - Model versioning is essential for traceability and controlled rollout - version the model together with relevant data, code/configuration and experiments.
        - Release record should state what changed, evaluation and segment results, expected business impact, known limits, owner and rollback condition.
        - Canary/limited rollout, rollback speed, model lineage and change visibility become critical if a new release behaves worse for one segment.
        - Examples: search ranking rollback and autonomous-driving software release gates, where readiness is more than a new model file.
    - API scaling needs intentional request management rather than treating all requests/customers identically.
        - Important controls: request queues, rate limits, timeouts, concurrency limits, retry/backoff, throttling and clear SLAs (Service Level Agreements).
        - Free, paid and enterprise customers may require different throughput/fairness/guarantee policies. One universal policy can be unfair or uneconomic.
        - Track queue depth, timeout rate, usage concentration by tenant and SLA violations; examples include Stripe reliability and Twilio multi-tenant throughput.
    - Scaling GenAI products adds specific challenges because prompts/context and model choices directly change the cost of serving each request.
        - Control options: prompt/context trimming, caching reusable results, routing simple tasks to smaller models, model tiering and limits on expensive calls.
        - Caching is useful only when the response remains relevant and safe; truncation should not remove information needed for a correct answer.
        - Monitor token consumption by workflow, cache hit rate, route mix, refusal/guardrail triggers, correction burden and trust signals.
        - GitHub Copilot routing and Notion AI usage bursts are examples for balancing user experience against inference cost.
    - Case study: scaling a generative AI customer-support assistant should consider accuracy, user trust, queue time, escalation/fallback, cost per resolved ticket and recoverability together.
        - PM decision: what service level to promise, what to route to cheaper models, when to require human review, and when rollout must be slowed.
    - The objective of scale is repeatable value with acceptable latency, reliability, economics and risk - not only growth in traffic.

➢ Lecture 12 - Ethics, Risk, and Responsible AI Products

- Responsible AI should be built into product decisions from discovery onward. It cannot be treated as a policy check only at the end of development.
- Bias in AI products can enter before the model makes any prediction:

        - Data bias - historical data is incomplete, skewed or does not represent every user group.
        - Labeling bias - label rules, proxy outcomes or human judgments encode inconsistent or unfair assumptions.
        - Product/policy bias - product eligibility, objectives, thresholds or business rules can disadvantage groups even when the model is technically accurate.
        - Deployment/workflow bias - availability, access, interfaces or human actions can result in unequal real-world outcomes.
        - Apple Card credit-limit controversy and facial-recognition performance gaps were examples for asking who benefits, who is harmed and what is invisible in an average metric.
        - Relevant evidence: outcome and error gaps by segment, complaint patterns, override rates and disagreement between humans and the AI.
    - Bias is therefore a product-design and governance concern, not only a post-launch public-relations concern.
        - The PM should ask where the earliest unfair assumption enters the workflow, including what target/label the organization has chosen.
    - Privacy - collect enough data to deliver and improve the product, but not every piece of data that might become useful later.
        - Data minimization - collect only adequate, relevant and necessary information for the defined purpose.
        - Consent and purpose limitation - explain what is collected and why; future reuse cannot be automatically assumed to be acceptable.
        - Handle sensitive attributes carefully; specify access rights, storage/retention periods, deletion controls and auditability.
        - Examples include Spotify event instrumentation, Meta advertising-data scrutiny and health-data-sharing concerns.
        - Data requirements should describe event coverage, freshness, privacy basis, retention, downstream uses and the decisions enabled by the data.
    - Generative AI hallucination - a model may produce a fluent, plausible but incorrect answer because generating likely text is not the same as verifying facts.
        - The risk is much higher when wrong output causes financial, medical, legal or safety consequences rather than minor inconvenience.
        - Mitigation: retrieval/grounding using trusted sources, verifiable citations, limited scope, quality checks, uncertainty messaging and human review.
        - Sources/citations reduce unsupported claims only when the underlying retrieval and evidence are actually correct; citations alone do not prove accuracy.
        - The product should choose among Answer -> Abstain/clarify -> Escalate based on confidence, consequence and available evidence.
        - Measure grounding success, citation coverage, escalations and high-severity wrong answers. A convincing incorrect answer is more dangerous than visible uncertainty.
    - Responsible AI design includes fairness, reliability/safety, privacy/security, inclusiveness, transparency and accountability.
        - Fairness: check similar cases/groups receive appropriate treatment; reliability: system works safely across intended conditions; privacy: data is protected.
        - Transparency: user understands what AI has done and its limitations; accountability: a named human/team owns decisions and consequences.

        - User recourse is important: explanation, ability to correct/override, appeal/escalate, and a clear recovery path after the AI is wrong.
        - Human-in-the-loop should specify when review is mandatory, reviewer responsibility, turnaround targets, overrides and escalation rather than adding an undefined manual step.
    - Governance mechanisms convert responsible AI principles into operating controls.
        - Internal review boards, product policies and documented exceptions should define which decisions require independent approval and which remain with the product team.
        - Good governance must be timely enough to help release decisions; too much paperwork without decision rights becomes a bottleneck.
        - Useful measures: review turnaround time, auditability, policy-exception rate, completeness of mitigation, incident closure and user recourse outcomes.
        - NIST AI Risk Management Framework provides a useful structure: Govern -> Map risks -> Measure risks -> Manage risks, carried out across the product lifecycle.
    - Risk assessment before launch should consider more than a simple total score:
        - Likelihood - chance of an adverse event; Severity - consequence; Detectability - how likely the harm will be noticed; Reversibility - ease of undoing it.
        - Assess inherent risk, mitigation controls, remaining/residual risk and whether a guarded rollout can safely contain exposure.
        - A risk matrix that appears precise but ignores reputation damage, vulnerable segments or poor reversibility is not sufficient.
        - Decision: Go -> Go with guardrails/human review -> Iterate/hold -> No-go, based on explicit thresholds and owner approval.
    - Hiring-algorithm case study - historical hiring patterns may carry past inequities into a new system; technical fairness tests alone are not enough if the workflow or policy itself is biased.
        - The product needs segment-level outcome checks, human review for consequential decisions, candidate recourse and clear ownership before launch.
    - The PM ultimately needs to identify which risks truly block launch, which can be mitigated through controls, and what evidence would justify any later expansion.